

Bessel Functions of Order Zero in a Shader

The Abramowitz–Stegun approximations used by `BesselNode`

BlenderMathAnim — `geometry_nodes/nodes.py`

Contents

1 Why an approximation at all	1
2 Small argument: $0 < x \leq 3$	1
3 Large argument: $3 \leq x < \infty$	2
4 How the two branches are joined	3
5 Accuracy actually achieved	3
6 Where this lives in the code	4

1 Why an approximation at all

Blender’s `Math` node offers the elementary operations and nothing else, and J_0 is not elementary: it cannot be assembled from $\sin$, $\cos$, $\exp$, $\log$ and the arithmetic operations in any finite number of steps. A shader that wants the field of a two-dimensional point source therefore has three options — a lookup texture, a truncated series, or a uniform approximation. The library uses the third, in the classical form tabulated by Abramowitz & Stegun, §9.4.1–9.4.3.¹

The functions in question are the two cylinder functions of order zero. They stand to a circular wave as $\cos$ and $\sin$ stand to a wave in one spatial dimension:

$$u(r, t) = \operatorname{Re} \left[H_0^{(1)}(kr) e^{-i\omega t} \right] = J_0(kr) \cos \omega t + Y_0(kr) \sin \omega t, \quad (1)$$

and the familiar $\sin(kr - \omega t)/\sqrt{r}$ is only what (1) becomes far from the source. Near it the two disagree completely: the elementary form diverges as $r^{-1/2}$ and puts its wavefronts in the wrong places, whereas the true field diverges only logarithmically.

The decisive property is the behaviour as $x \rightarrow \infty$. The large-argument branch below is written in *modulus and phase* form, so what the polynomials approximate is the slowly varying envelope and the phase *correction*, not the oscillation itself. It is therefore accurate at every x , however large. A lookup table cannot do that: it would have to be resampled whenever the wavelength or the extent of the surface changed, and it would have to store the oscillation.

2 Small argument: $0 < x \leq 3$

Both branches are polynomials in

$$t = \frac{x}{3}, \quad 0 < t \leq 1, \quad (2)$$

and, because J_0 is even and $Y_0 - \frac{2}{\pi} \ln(x/2)J_0$ is even, only even powers of t occur. Written traditionally:

¹M. Abramowitz and I. A. Stegun, *Handbook of Mathematical Functions*, Dover 1964, §9.4, pp. 369–370. The coefficients there are due to E. E. Allen (1954/1956).

J_0 , A&S 9.4.1

$$J_0(x) = 1 - 2.2499997 t^2 + 1.2656208 t^4 - 0.3163866 t^6 + 0.0444479 t^8 - 0.0039444 t^{10} + 0.0002100 t^{12} + \varepsilon, \quad (3)$$

with $|\varepsilon| < 5 \cdot 10^{-8}$. The stated range in the Handbook is $-3 \leq x \leq 3$; the code only ever evaluates it for $x > 0$.

Y_0 , A&S 9.4.2

Y_0 is logarithmically singular at the origin, and the singularity is not polynomial. It is split off explicitly, multiplied by J_0 — which is what makes Y_0 the outgoing half of the wave — and what remains is again an even polynomial in t :

$$Y_0(x) = \frac{2}{\pi} \ln\left(\frac{x}{2}\right) J_0(x) + 0.36746691 + 0.60559366 t^2 - 0.74350384 t^4 + 0.25300117 t^6 - 0.04261214 t^8 + 0.00427916 t^{10} - 0.00024846 t^{12} + \varepsilon, \quad (4)$$

with $|\varepsilon| < 1.4 \cdot 10^{-8}$, valid for $0 < x \leq 3$. The constant term is $\frac{2}{\pi}(\ln 2 + \gamma)$ to the precision carried, γ being Euler's constant. Note that the J_0 appearing in (4) is the *same* polynomial (3): the two small-argument branches share it.

3 Large argument: $3 \leq x < \infty$

Here the variable is reciprocal,

$$t = \frac{3}{x}, \quad 0 < t \leq 1, \quad (5)$$

and the two functions are written as one amplitude f_0 and one phase θ_0 , in the form (A&S 9.4.3)

$$J_0(x) = x^{-1/2} f_0(x) \cos \theta_0(x), \quad Y_0(x) = x^{-1/2} f_0(x) \sin \theta_0(x). \quad (6)$$

This is the whole point of the construction: the trigonometric function carries the oscillation exactly, and only the envelope and the phase shift are approximated. The polynomials are

$$f_0 = 0.79788456 - 0.00000077 t - 0.00552740 t^2 - 0.00009512 t^3 + 0.00137237 t^4 - 0.00072805 t^5 + 0.00014476 t^6 + \varepsilon, \quad (7)$$

with $|\varepsilon| < 1.6 \cdot 10^{-8}$, and

$$\theta_0 = x - 0.78539816 - 0.04166397 t - 0.00003954 t^2 + 0.00262573 t^3 - 0.00054125 t^4 - 0.00029333 t^5 + 0.00013558 t^6 + \varepsilon, \quad (8)$$

with $|\varepsilon| < 7 \cdot 10^{-8}$ (an absolute error *in the phase*, in radians).

The two leading constants are the asymptotics themselves: $0.79788456 \approx \sqrt{2/\pi}$ and $0.78539816 \approx \pi/4$, so that (6) reduces to the textbook

$$J_0(x) \sim \sqrt{\frac{2}{\pi x}} \cos\left(x - \frac{\pi}{4}\right), \quad Y_0(x) \sim \sqrt{\frac{2}{\pi x}} \sin\left(x - \frac{\pi}{4}\right), \quad x \rightarrow \infty, \quad (9)$$

with the polynomials in $t = 3/x$ supplying the correction. The approximation *is* the far-field wave plus a correction, which is why it does not degrade with distance.

4 How the two branches are joined

A shader has no branches: both sides of (3)/(4) and (6) are evaluated for every x , and one of them is then selected by an arithmetic mask. With

$$s = [x < 3] \in \{0, 1\}, \quad (10)$$

the result is $s \cdot (\text{small branch}) + (1 - s) \cdot (\text{large branch})$. Because the discarded branch is computed anyway, each side clamps its own argument before use:

$$x_{\text{small}} = \min(x, 3), \quad x_{\text{large}} = \max(x, 3). \quad (11)$$

Without the clamp the large- x branch would form $3/x$ at small x and multiply an infinity by the zero mask, producing NaN rather than nothing — $0 \cdot \infty$ is not 0 in IEEE 754.

Two further remarks on the translation into RPN:

- The polynomials are evaluated by Horner’s rule (`horner_rpn`). Beyond costing no **POWER** nodes — one multiply and one add per term — it is the better-conditioned order.
- The phase (8) is arranged as $\theta_0 = x - \pi/4 + t(a_1 + t(a_2 + \dots))$: the leading factor t is pulled out so that Horner runs over $a_1 \dots a_6$ and the bracket is multiplied by t once.
- The RPN vocabulary provides only the base-10 logarithm, so the $\frac{2}{\pi} \ln(x/2)$ of (4) is evaluated as $\frac{2}{\pi} \ln(10) \cdot \log_{10}(x/2)$, the change of base being folded into the constant that stands in front of it anyway.
- $Y_0(x) \rightarrow -\infty$ as $x \rightarrow 0$. The caller is responsible for holding x off the origin (a `max`), which physically means a source of finite radius rather than a point.

5 Accuracy actually achieved

Evaluated in double precision against `scipy.special` on $4 \cdot 10^6$ points of $(0, 400]$:

	$\max \Delta J_0 $	$\max \Delta Y_0 $
$0 < x < 3$	$4.9 \cdot 10^{-8}$	$2.4 \cdot 10^{-8}$
$3 \leq x \leq 400$	$1.2 \cdot 10^{-8}$	$9.1 \cdot 10^{-9}$
overall	$4.9 \cdot 10^{-8}$	$2.4 \cdot 10^{-8}$

Both figures sit below $1.2 \cdot 10^{-7}$, the resolution of the `float32` the shader evaluates in. As far as Blender is concerned the approximation is exact; the error that survives evaluation is the rounding of x itself, not the approximation of J_0 . The error does not grow with x — the large-argument entries above would read the same on $(400, 4000]$ — because $t = 3/x$ shrinks and the oscillation is carried by the exact $\cos/\sin$.

6 Where this lives in the code

Formula	Name in <code>nodes.py</code>
(3) coefficients	<code>_J0_SMALL</code>
(4) coefficients (polynomial part)	<code>_Y0_SMALL</code>
(7) coefficients	<code>_BESSEL_F0</code>
(8) coefficients $a_1 \dots a_6$	<code>_BESSEL_TH</code>
$\pi/4$	<code>_QUARTER_PI</code>
$\frac{2}{\pi} \ln 10$	<code>_TWO_OVER_PI_LN10</code>
Horner evaluation	<code>horner_rpn()</code>
the RPN for both functions	<code>bessel_j0_y0_rpn()</code>
the cached node group	<code>bessel_node_group()</code>
the group node	<code>BesselNode</code>
the postfix operators <code>j0, y0</code>	<code>BESSEL_OPS</code>

In a formula the functions appear as ordinary unary postfix operators, so the circular wave (1) of amplitude `amp` reads

```
"amp,x,j0,wt,cos,*,x,y0,wt,sin,*,+,*"
```

provided `BESSEL_OPS` is passed as `custom_ops`. Each operator instances a shared, fake-user group, so every source in a scene uses one copy of the approximation. Calling `bessel_j0_y0_rpn()` directly inlines the arithmetic instead, which is cheaper when both functions are wanted for the same argument, because then the shared polynomials — (3), (7), (8) — are evaluated once rather than once per group: 88 math nodes inlined against $63 + 81 = 144$ through the two groups.